Efficient Convolutional Neural Networks on Raspberry Pi for Image Classification

Rui-Yang Ju[1], Ting-Yu Lin[2], Jia-Hao Jian[1] and Jen-Shiun Chiang[1*]

[1]Department of Electrical and Computer Engineering, Tamkang University
No.151, Yingzhuan Rd., Tamsui Dist., New Taipei City, 251301, Taiwan.
[2]Department of Engineering Science, National Cheng Kung University
No. 1 University Rd., East Dist., Tainan City, 70101, Taiwan.

*Corresponding author(s). E-mail(s): jsken.chiang@gmail.com;
Contributing authors: jryjry1094791442@gmail.com; tonylin0413@gmail.com;
207440222@gms.tku.edu.tw;

Abstract

With the good performance of deep learning in the field of computer vision (CV), the convolutional neural network (CNN) architectures have become main backbones of image recognition tasks. With the widespread use of mobile devices, neural network models based on platforms with low computing power are gradually being paid attention. However, due to the limitation of computing power, deep learning algorithms are usually not available on mobile devices. This paper proposes a lightweight convolutional neural network TripleNet, which can operate easily on Raspberry Pi. Adopted from the concept of block connections in ThreshNet, the newly proposed network model compresses and accelerates the network model, reduces the amount of parameters of the network, and shortens the inference time of each image while ensuring the accuracy. Our proposed TripleNet and other State-of-the-Art (SOTA) neural networks perform image classification experiments with the CIFAR-10 and SVHN datasets on Raspberry Pi. The experimental results show that, compared with GhostNet, MobileNet, ThreshNet, EfficientNet, and HarDNet, the inference time of TripleNet per image is shortened by 15%, 16%, 17%, 24%, and 30%, respectively. The detail codes of this work are available at https://github.com/RuiyangJu/TripleNet.

Keywords: edge computing platform, image classification, convolutional neural network, model acceleration, model compression, Raspberry Pi.

1 Introduction

With the rapid development of information technology, artificial intelligence has reached milestone achievements in recent years. However, the huge amount of digital data imposes a burden on computing and power consumption, reducing the model size (the number of parameters or weights of the model), is a hot research topic to improve computation efficiency and energy efficiency. Generally, an edge device needs a cloud server to complete the calculation of the model. Its disadvantage is that the data transmission cost between the edge device and the cloud server is relatively high.

In recent years, CV applications based on Raspberry Pi have begun to attract people's

attention. Bechtel [1] simulated an autonomous-driving car by using the webcam and Raspberry Pi 3. He performed semantic segmentation tasks on Raspberry Pi to test the performance of neural networks. Monitoring systems are critical to the safety of human life, and Khalifa [15] compared different monitoring systems applied on Raspberry Pi and evaluated their performance. On this basis, Khalifa [16] proposed a new CNN model to complete the human detection task on Raspberry Pi. Unmanned aerial vehicles (UAVs) can replace ordinary aircraft for search and rescue applications. The target detection task on the drone can be simulated on Raspberry Pi. Mesvan [22] tested the Single Shot Detector (SSD) model on Raspberry Pi Model B, proving that their drone has an optimal detection distance of 1 to 20 meters. Caballero [2] used a CNN model on a Raspberry Pi to classify wastes into categories such as "plastic bottles", "aluminum cans", and "paper and cardboard" to infer recyclable objects. However, mobile phones, mobile devices, and robotic systems are unable to use deep learning algorithms due to limited computing power. Luo [21] combined deep learning-based CV algorithms with Raspberry Pi by utilizing the computing power of cloud servers. Although this method implements the deep neural network running on the Raspberry Pi, the data transferring between the Raspberry Pi and the cloud server consumes a lot of power. Therefore, this work aims to compress and accelerate the neural network model, and proposes a new Raspberry Pi based backbone network to promote various CV applications on Raspberry Pi.

Model compression and model acceleration are hot topics in deep neural network research. Model compression can reduce the number of parameters of neural networks, while model acceleration can reduce the inference time of neural networks. The performance of neural network models can be improved by model compression and acceleration. In order to implement vision applications in mobile devices and embedded systems, Google proposed MobileNet [9], which replaces the standard convolution layers in VGGNet [27] with depthwise separable convolutions. A year later, Google proposed MobileNetV2 [26], which introduced a linear bottleneck between the layers of the original neural network and added shortcut connections between the bottlenecks. ShuffleNet [33] uses the concept of group convolution to group feature maps to reduce computation, and therefore it can also be used in mobile phones or embedded systems. In 2019, Google once again proposed a new neural network scaling method that uses compound coefficients to uniformly scale the depth, width, and image resolution of the network. Compared with the previous SOTA network model, EfficientNet [29] using this method not only improves the accuracy by 10%, but also reduces the number of parameters by 8.4 times. In 2020, HUAWEI proposed GhostNet [5], which uses the Ghost module to reduce the amount of computation. GhostNet is more accurate than MobileNetV3 [8] under similar computation. However, unlike the above methods, this work argues that reducing the connections between layers can also reduce the computation of the network model.

ResNet [7] builds a network model with residual learning as the main architecture, deepens the depth of the network, and achieves excellent performance in tasks such as image classification, object detection, and semantic segmentation. He *et al.*, the authors of ResNet, added the split-transform-merge strategy to ResNet after referring to GoogLeNet [28]. The newly proposed ResNeXt [31] has better performance with the same parameters and computation as ResNet. Zagoruyko *et al.* thought from a different direction, abandoned the narrow characteristics of the ResNet model, and proposed Wide-ResNet [32], which reduces the number of parameters and shortens the training time. DenseNet [12] passes the output of each layer to each subsequent layer to facilitate feature reuse. However, the characteristic of interconnection among all layers results in larger requirements of memory usage and overlong inference time in practical applications. In order to overcome the drawbacks of massive hardware requirement in DenseNet, HarDNet [3] was proposed and can reduce the connections between layers, reduce the number of data transfers, and reset the weights of layers to increase feature extraction and maintain model accuracy. ThreshNet [14] uses a threshold mechanism to determine the connection between layers, and resets the number of channels to obtain higher accuracy and faster inference time than that of HarDNet. However, both HarDNet and ThreshNet ignore the problems of excessively large parameters, and their applications on low-computing power platforms are limited. Based on the ThreshNet model architecture,

Table 1 Details of TripleNet Model

Model	#Params	Flops	Layers	Channel	Growth Rate
TripleNet-S	9.67M	4.17G	6, 16, 16, 16, 2	128, 192, 256, 320, 720	32, 16, 20, 40, 160
TripleNet-B	12.63M	4.29G	6, 16, 16, 16, 3	128, 192, 256, 320, 1080	32, 16, 20, 40, 160

TripleNet is proposed in this paper. This work reduces the number of parameters by improving the convolution layers in the blocks, and reduces the inference time per 100 images while improving the accuracy of the network model.

The contributions of TripleNet proposed in this paper are as follows:

1) This work proposes a new method for model compression and model acceleration. Three different convolution layers are set in different blocks, and different convolution layers use different connection methods. This variety of network architecture design reduces the computation of the network model, which is suitable for application on the platform of Raspberry Pi.
2) Different from using a cloud server to establish data transmission, this paper emphasizes the direct use of a suitable lightweight neural network on Raspberry Pi, which is more feasible with the diversified applications of embedded systems.

2 Related Work

Model compression is an important field for neural network research, and many research works have rarefied DenseNet through different methods. LogDenseNet [10] performs sparse connections, reduces the number of input channels from L^2 to $LlogL$, and increases the output channel width to recover the accuracy dropping from the connection pruning. SparseNet [20] utilizes the same sparse method as LogDenseNet, except that there is a fixed block output outputting $L+1$ layers for L layers. Both models require increased growth rates to maintain accuracy, without taking the issues of computing performance and power consumption into consideration, however this sparse method is worth referring to. ConDenseNet [11] introduces group operation in 1×1 convolution and prunes the weights at the beginning of training. In addition, ConDenseNet improves on DenseNet by performing dense connection of convolutional layers across blocks. Nevertheless, it replaces the convolution layers in DenseNet, which is desirable. PeleeNet [30] is a lightweight network variant based on Densenet, mainly for mobile devices. It proposes Stem Block to realize the downsampling of input images and the increase of channel number. This block provides stronger feature representation with less computational complexity.

The sparse connection method proposed by HarDNet [3] replaces the original dense connection with a harmonic dense connection scheme, and improves the output weight of the layer without sparse connection. Harmonic dense connection reduces the memory usage of the model and power consumption of hardware computing. Roy [23] proposed the Convolution-Depthwise-Pointwise (CDP) layer, a new means of interpolating using depthwise separable convolution, and applied this method to HarDNet to reduce parameters. ThreshNet [14] adopts the threshold mechanism by combining dense connection and harmonic dense connection, and resets the number of channels to improve the model accuracy. It proves that the combination of dense connection and harmonic dense connection has better performance.

3 Proposed Method

3.1 Conv Layers

3.1.1 Conv Layer 1

In order to connect all the layers with the same feature map size in the block, we fix the 3×3 convolution output feature map as the growth rate, and the specific value of the growth rate is shown in Table 1. To reduce the computational complexity, we use a 1×1 convolution, and the output feature of the same fixed convolution is $4 \times$ growth rate. As shown in Fig. 1, BN [13] and ReLU [4] are added before two convolutions, respectively. Conv Layer 1 adopts the dense connection method, and the input of the next convolution layer is the feature map of all layers:

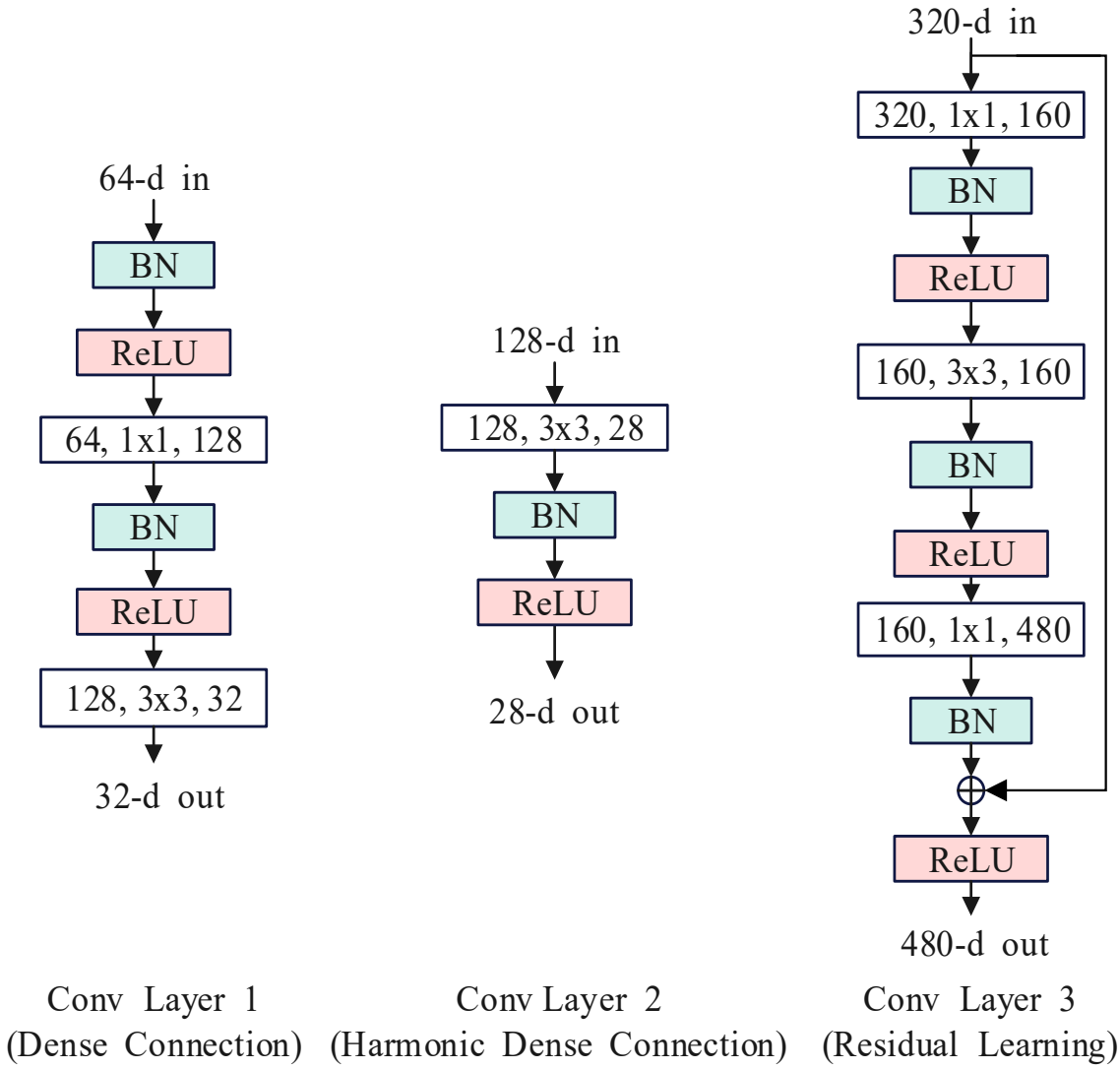

Fig. 1 Three convolution layers are used by TripleNet: (a) Conv Layer 1, using dense connections, (b) Conv Layer 2, using harmonic dense connections, (c) Conv Layer 3, using residual learning.

$$x_n = H_n([x_1, \cdots, x_{n-1}]) \tag{1}$$

3.1.2 Conv Layer 2

To reduce model memory usage and inference time, the convolution layers within the block are connected conditionally. Unlike Conv Layer 1, Conv Layer 2 adopts Conv-BN-ReLU, and only uses a single 3×3 convolution. Conv Layer 2 adopts the improved harmonic dense connection, and the fixed output of convolution with reserved input connection is $1.7 \times$ growth rate, as shown in

Algorithm 1 Conv Layer 2

Require: Stack 3×3 Conv Layer in each block
Ensure: 3×3 Conv - BN - ReLU

```
for layer n is even do
    for i in 1 to 5 do
        x = 2^i = 1
        y = n − x (x ≤ n)
        layer n connect to layer y
    end for
end for
for layer n is odd do
    layer n connect to layer y
end for
```

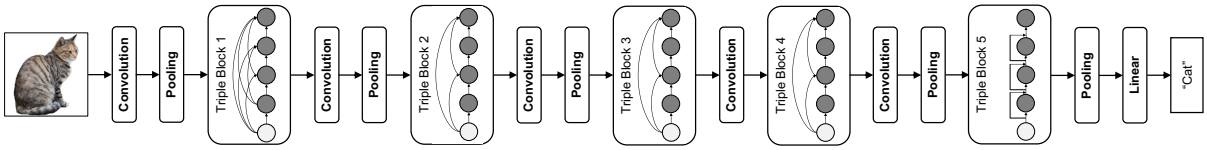

Fig. 2 The overall architecture of TripleNet.

Fig. 1, and the fixed output of convolution without input connection is the growth rate:

$$x_n = \begin{cases} H_n\left([x_{n-1}]\right), \text{ if } n\%2 = 1 \\ H_n\left([x_0, \cdots, x_{n-2^i}]\right), \text{ if } n\%2 = 0 \end{cases} \tag{2}$$

where the even-numbered layers are connected to the previous layers in the way of the Equation 2, while the odd-numbered layers are only connected to the previous layer. The specific connection method is shown in Algorithm 1.

3.1.3 Conv Layer 3

In order to reduce the number of parameters of the model and reduce the calculation time, the block depth is set to only 2 or 3. At the same time, to ensure the accuracy of the model, Conv Layer 3 is composed of three convolutions and uses residual learning. Since the 1 × 1 convolution is less computationally intensive, this architecture does not generate a large number of parameters. The first 1 × 1 convolution shrinks the input features by half, and the second 1 × 1 convolution triples the input features. The architecture is shown in Fig. 1:

$$x_{n+1} = x_n + \mathcal{F}\left(x_n, w_n\right) \tag{3}$$

3.2 Architecture

TripleNet consists of 5 blocks (each block is called a Triple-Block), and we propose three different convolution layers to construct these 5 blocks. After several experiments [10, 12, 14], Conv Layer 1 is the most effective approach to improve the accuracy of the model, but it takes up a lot of model memory, and therefore we only employ it in Triple-Block 1. Conv Layer 2 has a smaller memory usage, so we use it in Triple-Block 2, Triple-Block 3, and Triple-Block 4, which makes TripleNet retain its characteristics of smaller memory usage and suitable for edge devices. Conv Layer 3 uses residual learning, which has a smaller number of parameters. Because the input feature map of the last Triple-Block (Triple-Block 5) is large, using this convolution layer can ensure that TripleNet has a small number of parameters and a small amount of model calculation. By the mentioned scheme, the convolution layers used by the 5 blocks are determined, and the specific architecture is shown in Fig. 2.

The important characteristic of CNN architecture is to obtain a smaller size feature map by downsampling to extract the features for calculation; TripleNet applies the same thought as CNN. The 1 × 1 convolution and 2 × 2 average pooling layers before the block play the role of down-sampling, and we call the combination of the convolution and average pooling layers a transition layer.

3.3 Detailed Design

After extensive experiments, we propose two TripleNets, TripleNet-S and TripleNet-B, respectively, and the model details are shown in Table 1. TripleNet-S, ThreshNet-79 and HarDNet-68 are models of the similar order; TripleNet-B is of a higher model order. As we can see from Table 1, the difference between the two versions of the model is mainly in the last Triple-Block. The depth of the last Triple-Block of TripleNet-S is 2, while that of TripleNet-B is 3. As the depth increases, the numbers of channels, parameters, and the accuracy of the model all increase accordingly. Taking an image of size 224 × 224 in ImageNet as input, the feature map size extracted by TripleNet is shown in Table 2. A transition layer is added to the middle of each Triple-Block to obtain a smaller feature map of the input image, and the size of the feature map is 112^2, 56^2, 28^2, 14^2 and 7^2 respectively.

Table 2 Detailed Implementation Parameters

Layers	Output Size	TripleNet-S	TripleNet-B
First Layer	112^2 112^2 56^2	3×3 Convolution 3×3 Convolution 3×3 MaxPool	
Triple Block	56^2	[Conv Layer 1] $\times$ 6	[Conv Layer 1] $\times$ 6
Transition Layer	56^2 28^2	1×1 Convolution 2×2 MaxPool	
Triple Block 2	28^2	[Conv Layer 2] $\times$ 16	[Conv Layer 2] $\times$ 16
Transition Layer	28^2 14^2	1×1 Convolution 2×2 MaxPool	
Triple Block 3	14^2	[Conv Layer 2] $\times$ 16	[Conv Layer 2] $\times$ 16
Transition Layer	14^2	1×1 Convolution	
Triple Block 4	14^2	[Conv Layer 2] $\times$ 16	[Conv Layer 2] $\times$ 16
Transition Layer	14^2 7^2	1×1 Convolution 2×2 MaxPool	
Triple Block 5	7^2	[Conv Layer 3] $\times$ 2	[Conv Layer 3] $\times$ 3
Classification	1^2	AvgPool Linear	

Table 3 The details of the dataset

Dataset	Pixel	Training set	Test set	Classes
CIFAR-10	32×32	50,000	10,000	10
SVHN	32×32	73,257	26,032	10

4 Experiment

4.1 Environment Setup

4.1.1 Dataset

We list the training set, test set and classes of the datasets used in Table 3. CIFAR-10 [19] is a dataset consisting of color images of size 32 × 32, with 50,000 images for training and 10,000 images for testing, with a total of 10 classes. SVHN (Street View House Numbers) [24] is also a dataset consisting of color images of size 32 × 32, with 73,257 training images and 26,032 testing images. As shown in Figure 3, the image consists of some numbers, which are cropped from the house numbers on the street.

Fig. 3 Examples of images in the SVHN dataset.

4.1.2 Training

The selection of the optimization algorithm is the top priority of the training model. Even if the same network model is trained on the same dataset, using different optimization algorithms (SGD and Adam) leads to different training results. Gradient

Table 4 CIFAR-10 Classification Results and Model Architecture Parameters

	Inference Time (s)	Error (%)	Pre-Error (%)	Flops (G)	MAdd (G)	Memory (MB)	#Params (M)	MemR+W (MB)
TripleNet-S	4.06	13.05	12.75	4.17	8.32	90.25	9.67	238.36
ShuffleNet [33]	4.41	13.35	12.71	2.28	4.55	83.26	10.18	221.05
ThreshNet28 [14]	4.53	14.75	14.49	2.22	4.31	617.00	1.01	1009.03
TripleNet-B	6.51	12.97	12.62	4.29	8.57	91.33	12.63	755.07
MobileNetV2 [26]	6.74	14.06	13.95	2.42	4.75	384.78	2.37	98.84
GhostNet [5]	7.67	19.96	19.75	0.15	0.29	40.05	5.18	251.67
MobileNet [9]	7.68	16.12	15.00	2.34	4.63	230.84	3.32	474.13
ThreshNet 95 [14]	7.79	13.31	13.04	4.07	8.12	132.34	16.19	356.66
EfficientNet B0 [29]	8.54	13.40	13.06	1.51	2.99	203.74	3.60	421.54
HarDNet 85 [3]	9.25	13.89	13.62	9.10	18.18	74.65	36.67	313.42

All network models are implemented on the same platform Raspberry Pi 4. Raspberry Pi Time is the inference time per 100 images on Raspberry Pi 4. We denote pre-training by a **Pre** mark at the beginning of the error rate. (e.g., Pre-Error).

descent is currently the most widely used optimization algorithm in neural networks. In order to make up for the shortcomings of naive gradient descent, researchers have proposed a series of variant algorithms, which have gradually evolved from Stochastic Gradient Descent (SGD) to Adam [17]. In our experiments, all the networks are trained using Adam, which is a combination of RMSprop and Momentum. Similar to RMSprop using an exponential moving average for second-order momentum, Adam also uses an exponential moving average for first-order momentum. We set the coefficients used for computing running averages of gradient and its square to be 0.9 and 0.999, and eps (term added to the denominator to improve numerical stability) of 10^{-8}.

On CIFAR-10 and SVHN, we train models using batch size 64 for 200 and 60 epochs respectively. When training the model, the speed of gradient descent is controlled by the learning rate. The initial learning rate is set to 10^{-3}, and is divided by 5 at 37.5% and 75% of the total number of training epochs. The loss function is used to evaluate the degree to which the predicted value of the model is different from the true value, and we use the Cross-Entropy loss function.

4.1.3 Comparisons

This research uses the inference time per 100 images and test error rate to evaluate our algorithm and compare with many networks, including HarDNet, ThreshNet, ShuffleNet, MobileNetV1, MobileNetV2, GhostNet, and EfficientNet.

4.1.4 Testing

The trained network model is evaluated for performance on Raspberry Pi 4 Model B 4GB. The Raspberry Pi is evaluated using python 3.9, torch version 1.11.0.

4.2 Experiment Results

In Table 4, we show the test results of image classification on CIFAR-10 dataset. Sorted by the inference time per 100 images on Raspberry Pi, TripleNet-S has the shortest inference time of 40.6ms compared to other neural networks. Compared with ThreshNet 28, the number of parameters drops from 10.18M to 9.67M, and the error rate drops by 10%. Although ThreshNet28 has only 4 blocks, it still generates a large amount of computation because all layers in the last block use harmonic dense connections. TripleNet-S uses 5 blocks, but the last block uses the newly proposed convolution layer, which does not generate too much computation, but the network depth is improved. We think using this approach is more reasonable than simply reducing the depth of ThreshNet. Compared with other SOTA networks, TripleNet-B has the lowest error rate of 12.97%, and the inference time on Raspberry Pi is also lower than MobileNetV2. We think this is because the CIFAR-10 dataset has only 10 classes.

Table 5 SVHN Classification Results and Model Architecture Parameters

	Inference Time (s)	Error (%)	Flops (G)	MAdd (G)	Memory (MB)	#Params (M)	MemR+W (MB)
TripleNet-S	3.87	6.18	4.17	8.32	90.25	9.67	238.36
ThreshNet 28 [14]	4.15	6.34	2.28	4.55	83.26	10.18	221.05
ShuffleNet [33]	4.56	6.34	2.22	4.31	617.00	1.01	1009.03
MobileNetV2 [26]	5.21	6.68	2.42	4.75	384.78	2.37	755.07
GhostNet [5]	6.62	8.36	0.15	0.29	40.05	5.18	98.84
TripleNet-B	6.62	5.67	4.29	8.57	91.33	12.63	251.67
MobileNet [9]	7.38	7.36	2.34	4.63	230.84	3.32	474.13
ThreshNet 95 [14]	7.86	6.89	4.07	8.12	132.34	16.19	356.66
EfficientNet B0 [29]	9.08	6.12	1.51	2.99	203.74	3.60	421.54
HarDNet 85 [3]	9.48	6.91	9.10	18.18	74.65	36.67	313.42

All network models are implemented on the same platform Raspberry Pi 4. Raspberry Pi Time is the inference time per 100 images on Raspberry Pi 4.

If the network depth is too deep, it would lead to overfitting problems. The experimental results in Table 4 prove that for the small dataset CIFAR-10 TripleNet is more suitable for inference on mobile devices such as Raspberry Pi. To demonstrate the above conclusions, we conduct image classification tasks on the small dataset SVHN. The test results are shown in Table 5. TripleNet-S performs image classification on Raspberry Pi, and the inference time per 100 images is still shorter than other neural networks, and the accuracy is not lower than ThreshNet and ShuffleNet. We emphasize that the combination of the three convolution layers reduces the number of parameters, which is an important reason for reducing inference time. The inference time of TripleNet-B is similar to GhostNet, and the error rate is lower than other SOTA neural networks. This proves that for image classification tasks, the TripleNet model performance is better than other neural networks on small datasets, and it is more suitable for running on mobile devices with limited computing power.

On CIFAR-10 dataset, we train all networks for 200 epochs, since the training loss at the 200th epoch is already close to 0.001. However, we found that DenseNet [12] used 300 epochs when training all networks on CIFAR-10. Therefore, in order to make the experiment more convincing, we pre-trained all the networks, so that the model can have a higher accuracy in training 200 epochs. As shown in Table 4, Pre-Error represents the test accuracy of the model with pre-training. With pre-training, the error rate of all models decreased by between 1% and 7%, and the model with higher testing error rate decreased more with pre-training. For example, the testing error rate of MobileNet decreased from 16.12% to 15% with pre-training. With pre-training, our proposed network models, TripleNet-S and TripleNet-B, drop by 2.3% and 2.7%, respectively. It is evident that TripleNet-B still maintains the lowest testing error rate 12.62% after pre-training among all models.

5 Conclusion

As a CNN backbone network, TripleNet can be applied in different scenarios, such as the application to Mask R-CNN [6] algorithm architecture to complete instance segmentation, the application to YOLOv3 [25] algorithm architecture to complete object detection, the application to Panoptic FPN [18] algorithm architecture to complete the panoptic segmentation. Compared with HarDNet and ThreshNet, TripleNet has a smaller number of model parameters and can complete inference in a shorter time on Raspberry Pi. TripleNet is a network architecture designed to achieve highly efficient performance with limited computing power, which is more suitable for real-life scenarios. As a common embedded system, Raspberry Pi can be used in many scenarios. We emphasize that inferring a neural network directly on Raspberry Pi is more efficient than using the cloud to build transmissions. The premise is that the neural network has a small amount of computation. TripleNet has efficient model performance and is the result

of model compression and model acceleration on ThreshNet. TripleNet outperforms ThreshNet in terms of parameters, accuracy, and inference time.

Different from the large dataset of ImageNet, small datasets, such as CIFAR-10 and SVHN, do not require computationally expensive neural networks, which sometimes lead to poor results due to overfitting. Therefore, for image classification tasks with smaller datasets on Raspberry Pi, TripleNet is more suitable than other SOTA neural networks.

6 Declarations

6.1 Funding

The authors did not receive support from any organization for the submitted work.

6.2 Competing interests

The authors have no financial or proprietary interests in any material discussed in this article.

6.3 Ethics approval

This research does not involve human participants and/or animals.

References

[1] Bechtel, M.G., McEllhiney, E., Kim, M., Yun, H.: Deeppicar: A low-cost deep neural network-based autonomous car. In: 2018 IEEE 24th international conference on embedded and real-time computing systems and applications (RTCSA), pp. 11–21. IEEE (2018)

[2] Caballero, J., Vergara, F., Miranda, R., Serracín, J.: Inference of recyclable objects with convolutional neural networks. arXiv preprint arXiv:2104.00868 (2021)

[3] Chao, P., Kao, C.Y., Ruan, Y.S., Huang, C.H., Lin, Y.L.: Hardnet: A low memory traffic network. In: Proceedings of the IEEE/CVF international conference on computer vision, pp. 3552–3561 (2019)

[4] Glorot, X., Bordes, A., Bengio, Y.: Deep sparse rectifier neural networks. In: Proceedings of the fourteenth international conference on artificial intelligence and statistics, pp. 315–323. JMLR Workshop and Conference Proceedings (2011)

[5] Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: More features from cheap operations. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 1580–1589 (2020)

[6] He, K., Gkioxari, G., Dollár, P., Girshick, R.: Mask r-cnn. In: Proceedings of the IEEE international conference on computer vision, pp. 2961–2969 (2017)

[7] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778 (2016)

[8] Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., et al.: Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision, pp. 1314–1324 (2019)

[9] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)

[10] Hu, H., Dey, D., Del Giorno, A., Hebert, M., Bagnell, J.A.: Log-densenet: How to sparsify a densenet. arXiv preprint arXiv:1711.00002 (2017)

[11] Huang, G., Liu, S., Van der Maaten, L., Weinberger, K.Q.: Condensenet: An efficient densenet using learned group convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 2752–2761 (2018)

[12] Huang, G., Liu, Z., Van Der Maaten, L., Weinberger, K.Q.: Densely connected convolutional networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 4700–4708 (2017)

[13] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: International conference on machine learning, pp. 448–456. PMLR (2015)

[14] Ju, R.Y., Lin, T.Y., Jian, J.H., Chiang, J.S., Yang, W.B.: Threshnet: An efficient densenet using threshold mechanism to reduce connections. arXiv preprint arXiv:2201.03013 (2022)

[15] Khalifa, A.F., Badr, E., Elmahdy, H.N.: A survey on human detection surveillance systems for raspberry pi. Image and Vision Computing **85**, 1–13 (2019)

[16] Khalifa, A.F., Elmahdy, H.N., Badr, E.: Real-time human detection model for edge devices. arXiv preprint arXiv:2111.10653 (2021)

[17] Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)

[18] Kirillov, A., Girshick, R., He, K., Dollár, P.: Panoptic feature pyramid networks. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 6399–6408 (2019)

[19] Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images (2009)

[20] Liu, W., Zeng, K.: Sparsenet: A sparse densenet for image classification. arXiv preprint arXiv:1804.05340 (2018)

[21] Luo, Z., Small, A., Dugan, L., Lane, S.: Cloud chaser: real time deep learning computer vision on low computing power devices. In: Eleventh International Conference on Machine Vision (ICMV 2018), vol. 11041, pp. 743–750. SPIE (2019)

[22] Mesvan, N.: Cnn-based human detection for uavs in search and rescue. arXiv preprint arXiv:2110.01930 (2021)

[23] Miles, R., Mikolajczyk, K.: Compressing local descriptor models for mobile applications. In: ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 1895–1899. IEEE (2021)

[24] Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A.Y.: Reading digits in natural images with unsupervised feature learning (2011)

[25] Redmon, J., Farhadi, A.: Yolov3: An incremental improvement. arXiv preprint arXiv:1804.02767 (2018)

[26] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 4510–4520 (2018)

[27] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)

[28] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., Rabinovich, A.: Going deeper with convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1–9 (2015)

[29] Tan, M., Le, Q.: Efficientnet: Rethinking model scaling for convolutional neural networks. In: International conference on machine learning, pp. 6105–6114. PMLR (2019)

[30] Wang, R.J., Li, X., Ling, C.X.: Pelee: A real-time object detection system on mobile devices. Advances in neural information processing systems **31** (2018)

[31] Xie, S., Girshick, R., Dollár, P., Tu, Z., He, K.: Aggregated residual transformations for

deep neural networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1492–1500 (2017)

[32] Zagoruyko, S., Komodakis, N.: Wide residual networks. arXiv preprint arXiv:1605.07146 (2016)

[33] Zhang, X., Zhou, X., Lin, M., Sun, J.: Shufflenet: An extremely efficient convolutional neural network for mobile devices. In: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 6848–6856 (2018)